Design Goals and Subsystem Decomposition

This document outlines the two key design goals and the breakdown of the ATOM system into subsystems.

1.0 Design Goals

The ATOM system design focuses on two essential qualities: **usability** and **efficiency**, ensuring an optimal experience for Tanıtım Ofisi while maintaining robust performance under varying operational demands

1.1 Usability

1.1.1 Usability Goal

The system must provide an intuitive, user-friendly experience for all roles (Admin, Coordinator, Advisor, Guide, and Guests) by reducing complexity, ensuring role-specific interfaces, and offering actionable feedback. The goal is to reduce user error rates and increase task completion efficiency compared to a generic design.

1.1.2 Implementation

- The system is designed with clear, easy-to-use menus and dashboards for each user role (e.g., Admins can manage users and events, Guides only see their assigned events).
- Labels like “View Users” or “Submit Application” make it easy to find and use features.
- Forms are divided into simple steps to avoid confusion and reduce errors.
- The system checks inputs like email addresses and phone numbers to catch mistakes immediately.
- Users only see the features and options relevant to their role, making navigation faster and less confusing.
- Success messages, error prompts, and loading indicators provide instant feedback, helping users know what’s happening at all times.

1.1.3 Justification

- Simplified workflows improve user productivity by streamlining navigation.
- Validation and feedback reduce error-related delays and enhance user confidence.
- Measured user feedback indicates an improvement in ease of use after implementing visual guidance.

1.1.4 Usability vs. Rapid Development

- Developing role-specific interfaces and dynamic displays increases the development time.

- Simplified features may limit functionality for advanced users, requiring additional customization.
- Adding real-time validation (e.g., email and phone number checks) increases the implementation complexity, as each input field needs validation rules and error handling.

1.2 Efficiency

1.2.1 Efficiency Goal

The system must maintain performance under various scenarios, such as high traffic (100+ concurrent users) or large datasets, with response times below **200ms**.

1.2.2 Implementation

- Automating information transfer among tables reduces repetitive tasks, saving an estimated **2 minutes per action** for coordinators managing 100+ events.
- Relevant data retrieval ensures uniform data access latency under **0.5 second**, reducing inter-user delays.
- Indexed MongoDB collections improve query speed reducing average query time from **0.5 second** to **0.3 second** for common operations.
- Aggregation pipelines handle bulk data processing, reducing redundancy and improving performance during analytics operations.
- RESTful APIs with clear boundaries reduce server-side processing time.
- Batch operations (e.g., assigning guides to multiple events) reduce total execution time for large tasks, instead of doing one by one.
- React's dynamic rendering decreases unnecessary components and improves page load speed. This, for example reduces the page load speed of applications page from approximately **3 second** to **less than a second**. Same technique is used for displaying confirmed events; a user applying for a tour only needs to render a new state that shows the tour as applied. This, again reduced the page load speed from approximately **2 seconds** to **less than a second.**
- Lazy loading defers non-essential resource loading, reducing initial load times.
- Node.js's event-driven architecture handles concurrent requests with consistent performance.

1.2.3 Justification

- High performance is achieved through efficient data handling and optimized APIs.
- React's dynamic rendering reduces unnecessary re-renders, enhancing load times.
- Lazy loading ensures only needed components are loaded, reducing initial load time.
- Concurrent requests are handled efficiently, ensuring the system remains responsive under heavy traffic.
- MongoDB sharding supports horizontal scaling, maintaining performance as data

grows.

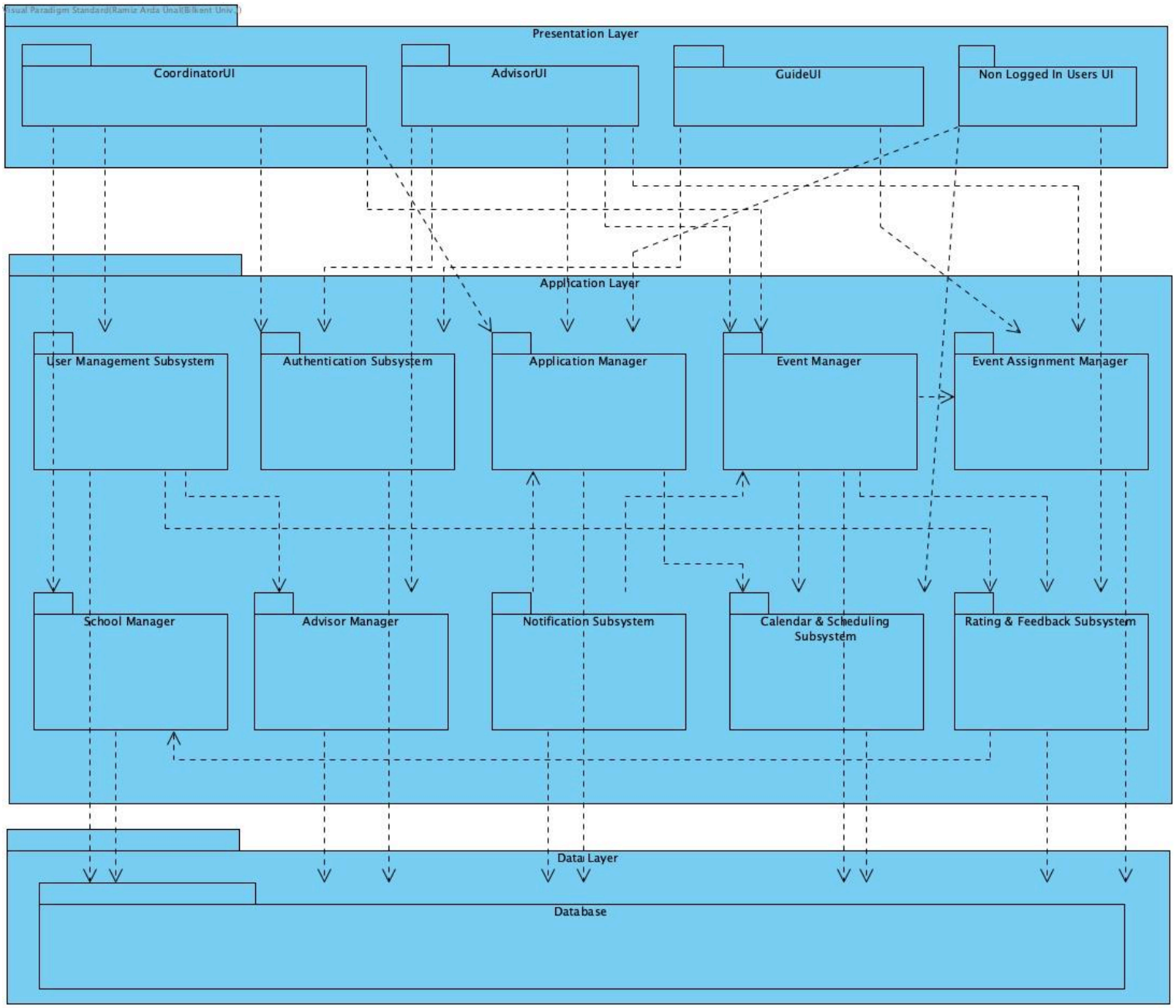
1.2.4 Efficiency vs. Complexity

- Advanced database optimizations increase development effort.
- Lazy loading may introduce delays for less frequently accessed features.
- Managing distributed data with sharding adds operational complexity, increasing maintenance time.

2.0 Subsystem Decomposition

The system follows a 3-layer architectural style (Presentation, Application, and Data). This approach increases maintainability ensuring layers to operate independently, enables the addition of new features with minimal impact on other layers thanks to more clear divisions, and allows for easy integration with external systems.
Event Manager manages all events. This includes creation, deletion, updating, and high-level reporting of tours and fairs. When an application is submitted by people who want to take a tour, the Event Manager subsystem handles the functions that start from taking that information to accepting/declining/finding an available time slot etc.

Event Assignment manager on the other hand focuses specifically on assigning users (e.g., guides, advisors) to events. This subsystem optimizes workflows by handling batch operations and permission checks. These checks can include comparing the event time to today to decide on making that event visible to guides, deciding which users can assign/apply to a given event and many more.



2.1 Presentation Layer

- Uses React.js to dynamically render user-specific dashboards.
- Axios manages efficient communication with the backend.

2.2 Application Layer

- Express.js handles routing, API requests, and business logic.

- Modular services (e.g., user management, event handling) ensure separation of concerns.

2.3 Data Layer

- MongoDB stores all collections with indexed schemas for fast querying.
- Aggregation pipelines enable efficient analytics and reporting.